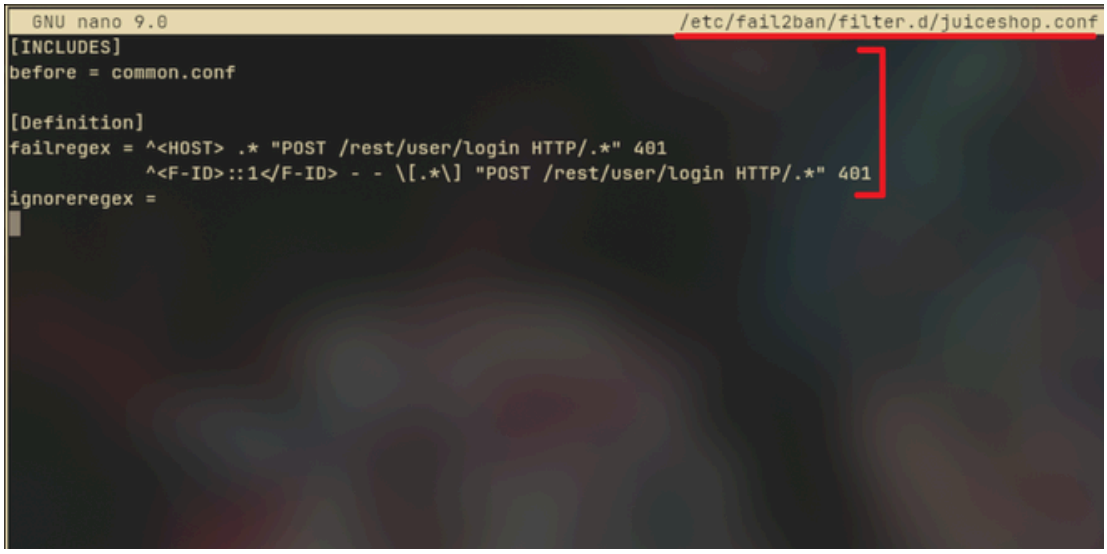
INTERNSHIP METADATA

- **Intern Name: Syed Mohib Ali Shah**
- **Intern ID: DHC-495**
- **Project Domain: Cybersecurity & Server Hardening**
- **Submission Date: June 9, 2026**
- **Status: [Successfully Implemented]**
-

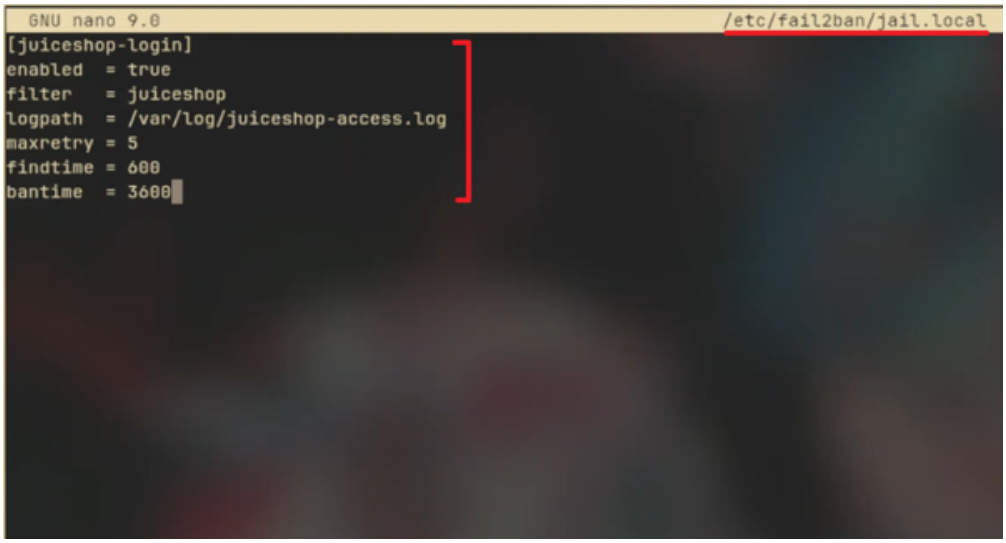


(Figure 2.1.1)

[INCLUDES]
before = common.conf

[Definition]
failregex = ^<HOST> .* "POST /rest/user/login HTTP/.*" 401
^<F-ID>::1</F-ID> - - \[.*\] "POST /rest/user/login HTTP/.*" 401
ignoreregex =

A corresponding jail was configured at /etc/fail2ban/jail.local:



(Figure 2.1.2)

Week 4 - Advanced Threat Detection & Web Security Enhancements

1.0 Overview

The objective of Week 4 was to implement advanced security measures on the OWASP Juice Shop application, detect threats in real-time, and harden the API endpoints against common attack vectors. The three main areas of focus were Intrusion Detection & Monitoring, API Security Hardening and Security Headers & Content Security Policy (CSP) Implementation

2.0 Intrusion Detection & Monitoring

2.1 What is Intrusion Detection?

Intrusion Detection refers to the process of monitoring system or network activity for malicious behaviour or policy violations. When suspicious activity is detected - such as repeated failed login attempts - the system can alert administrators or automatically block the offending source.

2.2 Setting Up Fail2Ban

Fail2Ban is an open-source intrusion prevention framework that monitors log files and bans IP addresses that show signs of malicious behaviour, such as repeated failed authentication attempts.

Fail2Ban was already installed on the system and was confirmed using:

sudo systemctl status fail2ban

A custom filter was created at */etc/fail2ban/filter.d/juiceshop.conf* to detect failed login attempts from the Juice Shop access log:

[juiceshop-login]
enabled = true
filter = juiceshop
logpath = /var/log/juiceshop-access.log
maxretry = 5
findtime = 600
bantime = 3600

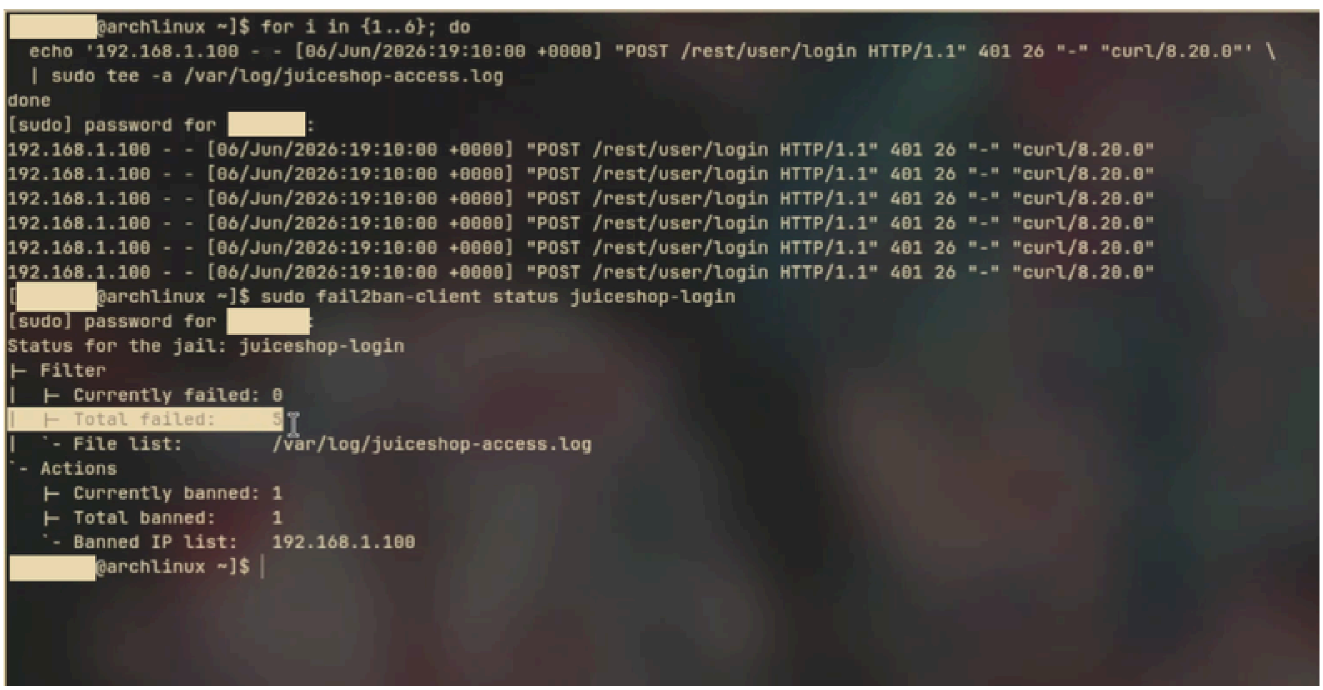
This configuration monitors the Juice Shop access log for repeated 401 responses on the login endpoint. If an IP address generates 5 or more failed login attempts within 600 seconds (10 minutes), it is automatically banned for 3600 seconds (1 hour).

2.3 Verifying Fail2Ban

To verify the jail was active and monitoring correctly, the following command was used:

sudo fail2ban-client status juiceshop-login

To test the detection capability, simulated failed login entries were appended to the monitored log:



(Figure 2.3.1)

```
for i in {1..6}; do
  echo '192.168.1.100 - - [06/Jun/2026:19:10:00 +0000] "POST /rest/user/login HTTP/1.1"
401 26 "-" "curl/8.20.0"' \
  | sudo tee -a /var/log/juiceshop-access.log
done
```

After Fail2Ban processed the log, the IP 192.168.1.100 was successfully banned:

```
Status for the jail: juiceshop-login
|- Filter
| |- Currently failed: 0
| |- Total failed:     5
| `- File list:        /var/log/juiceshop-access.log
`- Actions
   |- Currently banned: 1
   |- Total banned:     1
   `- Banned IP list: 192.168.1.100
```

2.4 Layered Defence Note

During testing, it was observed that the express-rate-limit middleware (configured in Section 3.1) intercepts brute-force login attempts before they generate 401 responses, returning 429 (Too Many Requests) instead. This means Fail2Ban serves as a complementary second layer - it handles 401-based attacks from sources not yet rate-limited, while express-rate-limit handles high-volume attacks first. Together, they form a layered defence system.

3.0 API Security Hardening

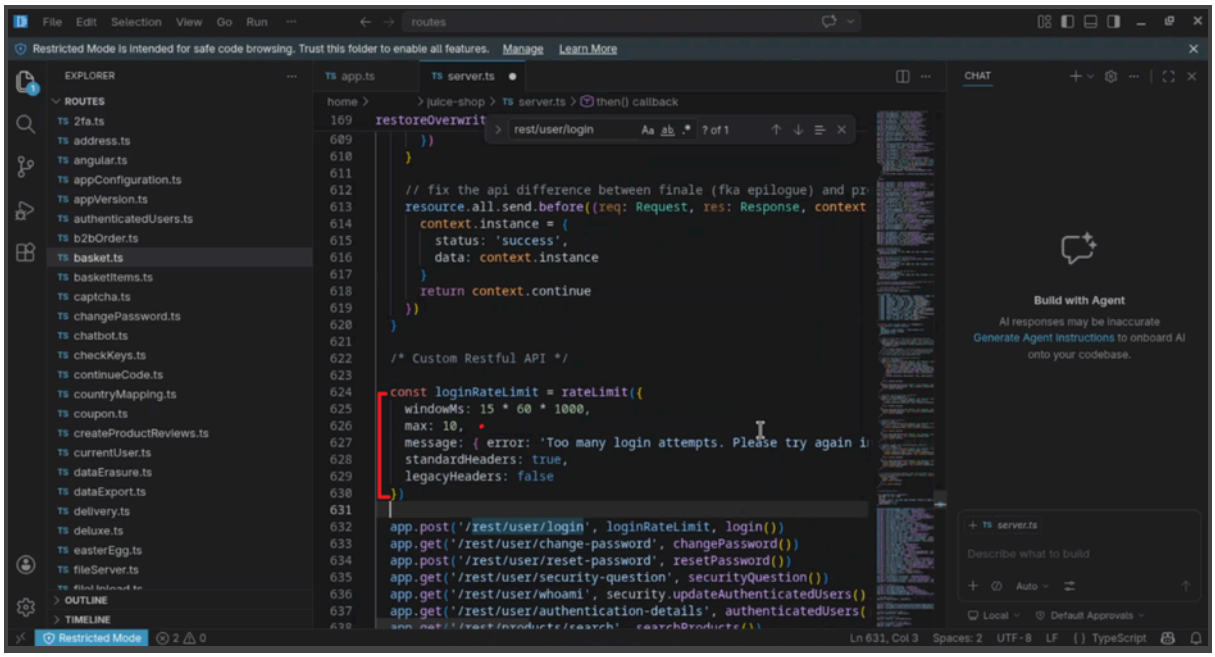
3.1 Rate Limiting

3.1.1 What is Rate Limiting?

Rate limiting is a technique used to control the number of requests a client can make to a server within a defined time window. Without rate limiting, attackers can perform brute-force attacks by attempting thousands of password combinations in a short period.

3.1.2 Implementation

The express-rate-limit package was used to apply rate limiting to the login endpoint. The following configuration was added to server.ts:



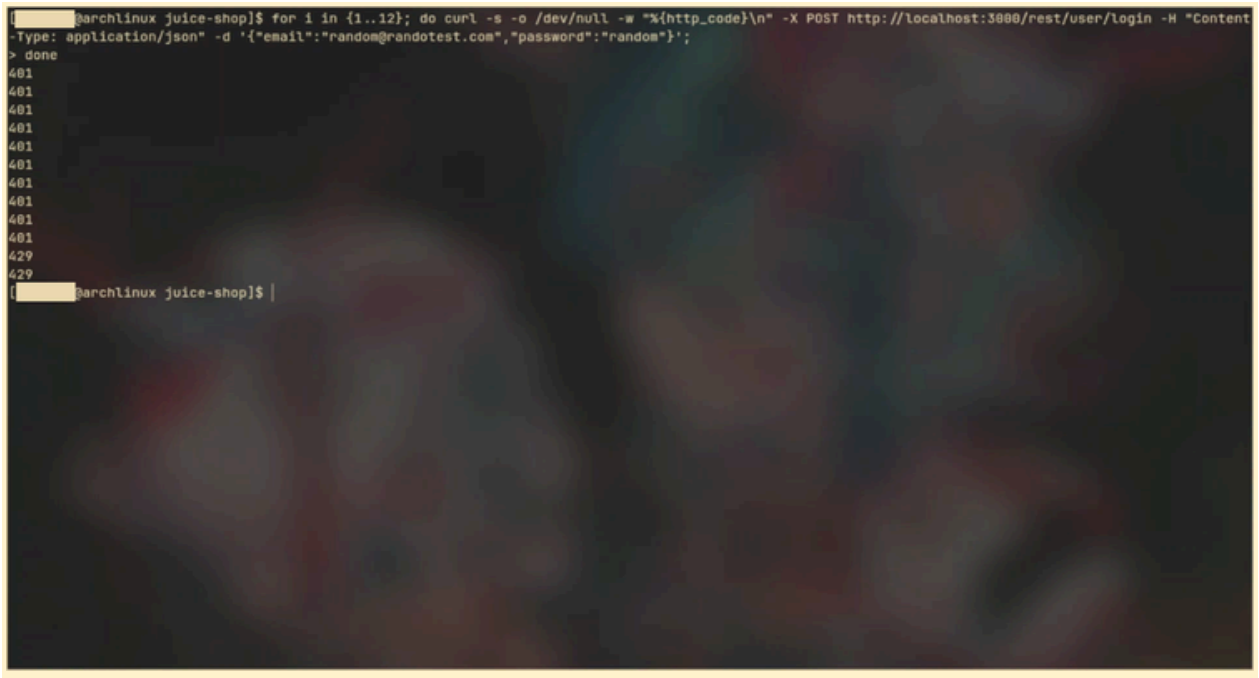
(Figure 3.1.1)

```
const loginRateLimit = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minute window
  max: 10,                  // maximum 10 attempts per window
  message: { error: 'Too many login attempts. Please try again in 15 minutes.' },
  standardHeaders: true,
  legacyHeaders: false
}))
```

This limits each IP address to 10 login attempts every 15 minutes.

3.1.3 Verification

To verify the rate limiter was functioning correctly, the following command was used to send 12 consecutive login requests:



(Figure 3.1.2 - Terminal output showing 401 responses changing to 429 after the 10th attempt)

```
for i in {1..12}; do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -X POST http://localhost:3000/rest/user/login \
    -H "Content-Type: application/json" \
    -d '{"email":"randomtest@randotest.com","password":"random"}'
done
```

Thefirst 10 requests returned 401 *Unauthorized*, and requests 11 and 12 returned 429 *Too Many Requests*, confirming that rate limiting was working as expected. I accidentally wrote “randotest” instead of “randomtest”, but the point still stands.

3.2 CORS Configuration

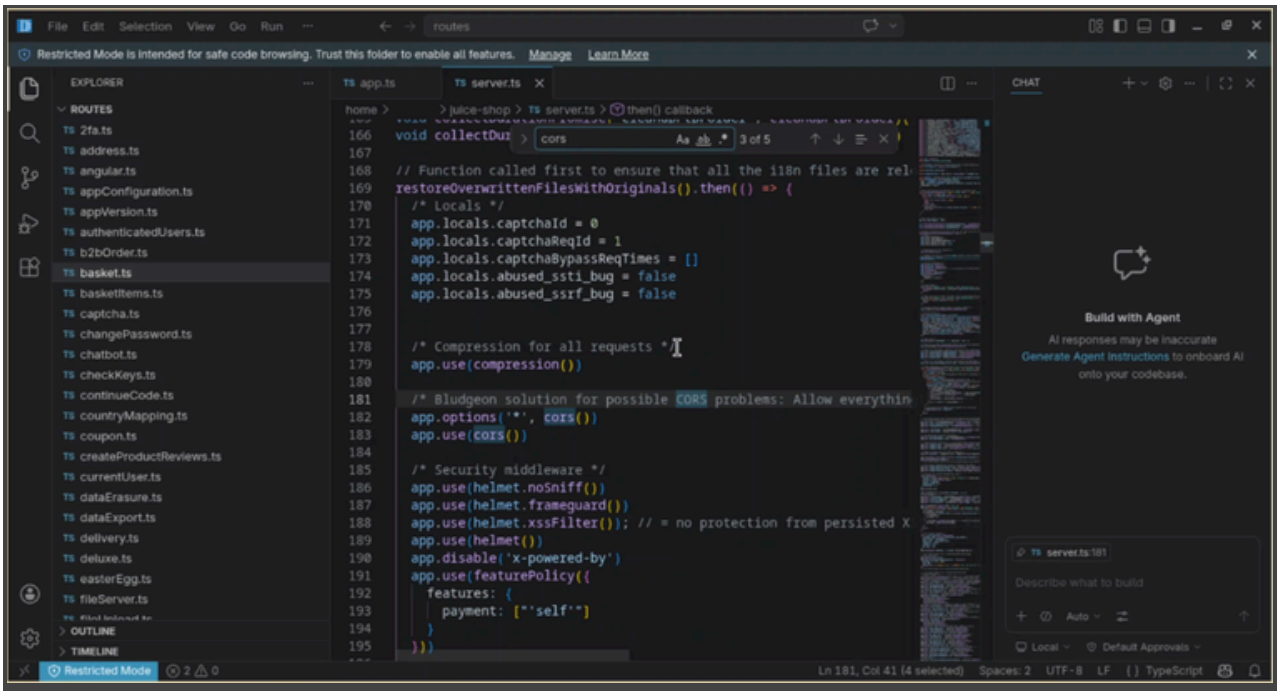
3.2.1 What is CORS?

Cross-Origin Resource Sharing (CORS) is a browser security mechanism that controls which external domains are permitted to make requests to a web server. A misconfigured CORS policy that allows all origins (*) can expose APIs to cross-site request abuse.

3.2.2 Implementation

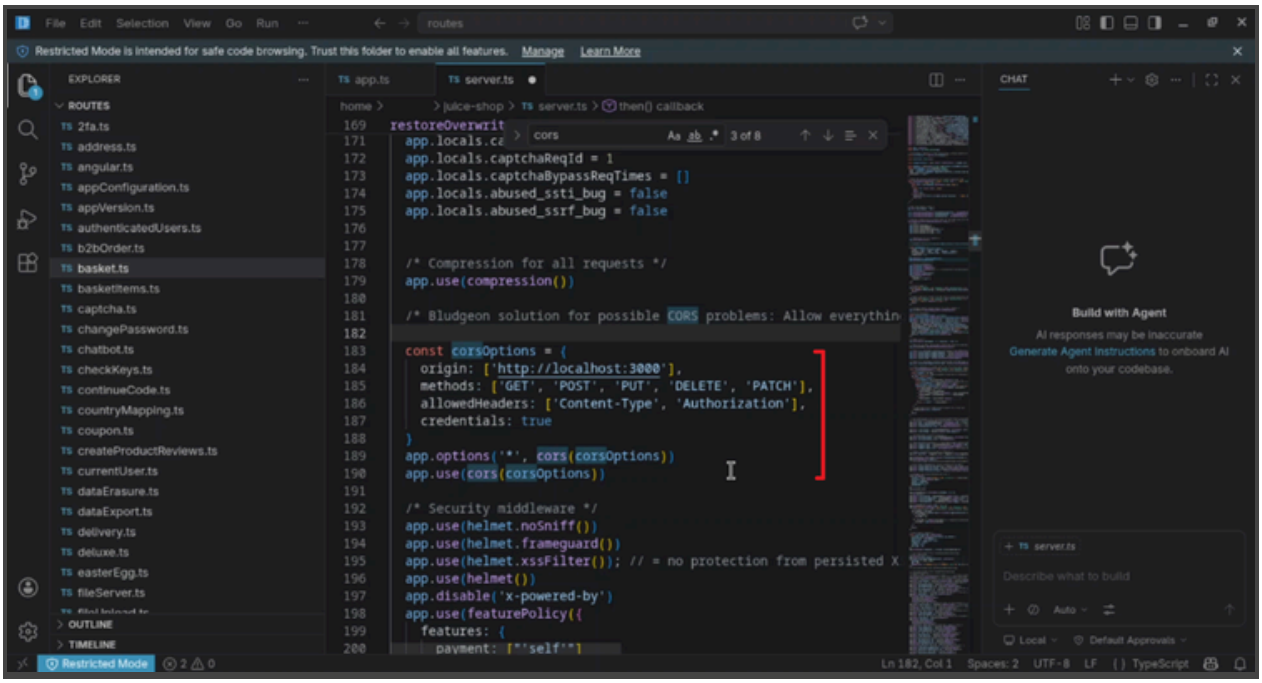
The default Juice Shop configuration used an open CORS policy that allowed all origins:

app.options('', cors())*
app.use(cors())



(Figure 3.2.1)

This was replaced with a restricted allowlist:



(Figure 3.2.2)

```
const corsOptions = {
  origin: ['http://localhost:3000'],
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true
}
app.options('*', cors(corsOptions))
app.use(cors(corsOptions))
```

3.2.3 Verification

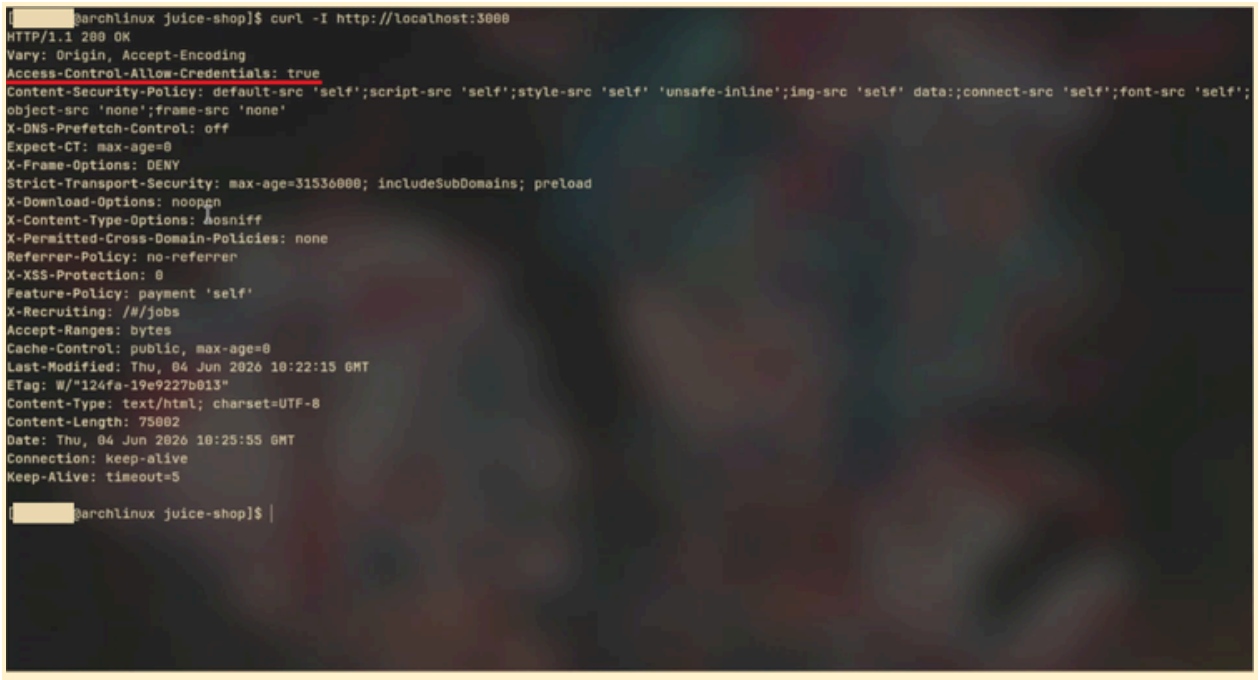
The change was verified by inspecting the response headers:

*Before: Access-Control-Allow-Origin: **

(Figure3.2.3)

After:

Access-Control-Allow-Origin: true



(Figure 3.2.4)

3.3 API Authentication

Juice Shop uses JWT (JSON Web Token) based authentication enforced via the security.isAuthorized() middleware. All sensitive endpoints - including /rest/basket, /api/Users, /api/Cards, and /b2b/v2 - require a valid JWT token in the Authorization: Bearer header. Unauthenticated requests are rejected with 401 Unauthorized.

4.0 Security Headers & CSP Implementation

4.1 What are Security Headers?

HTTP security headers are directives sent by the server to the browser that instruct it how to behave when handling the site's content. They protect against a wide range of attacks including clickjacking, MIME sniffing, XSS, and protocol downgrade attacks.

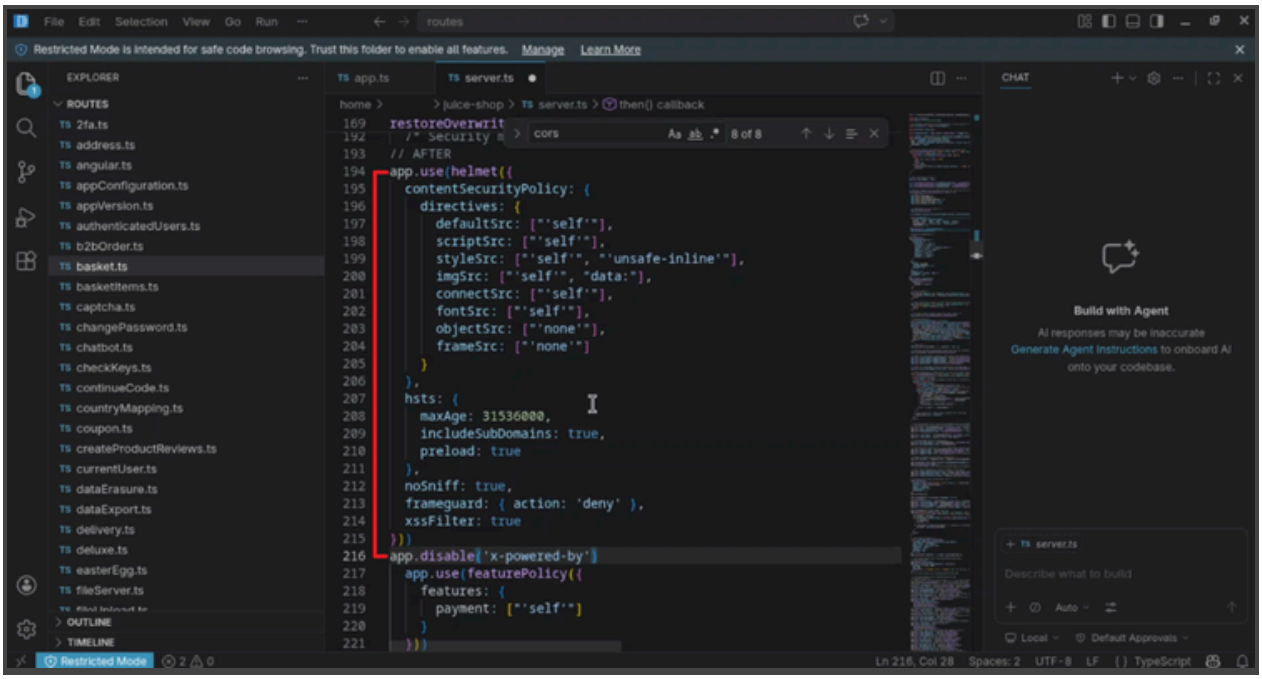
4.2 Content Security Policy (CSP)

4.2.1 What is CSP?

Content Security Policy (CSP) is an HTTP response header that allows a server to declare which dynamic resources are allowed to load. It is one of the most effective defences against Cross-Site Scripting (XSS) attacks, as it prevents the browser from executing scripts from unauthorised sources.

4.2.2 Implementation

The existing Juice Shop code used individual helmet middleware calls without a unified CSP configuration. This was replaced with a comprehensive helmet() configuration in server.ts:



(Figure 4.2.1)

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc:     ["'self'", "data:"],
      connectSrc: ["'self'"],
      fontSrc:    ["'self'"],
      objectSrc: ["'none'"],
      frameSrc: ["'none'"],
      frameAncestors: ["'none'"],
      formAction: ["'self'"]
    }
  },
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
```

```
    preload: true
  },
  noSniff: true,
  frameguard: { action: 'deny' },
  xssFilter: true
}))
app.disable('x-powered-by')
```

Note on style-src 'unsafe-inline': The Angular frontend requires inline styles to render correctly. Removing this directive causes the UI to break. This is documented as a known trade-off between strict CSP compliance and application functionality.

4.3 HTTP Strict Transport Security (HSTS)

4.3.1 What is HSTS?

HTTP Strict Transport Security (HSTS) is a security policy that instructs browsers to only communicate with the server over HTTPS, preventing protocol downgrade attacks and cookie hijacking.

4.3.2 Implementation

HSTS was configured as part of the helmet configuration above with a maxAge of 31,536,000 seconds (1 year), includeSubDomains, and preload flags set.

4.4 Verification

After rebuilding and restarting the server, all security headers were verified using:

curl -I http://localhost:3000

The following headers were confirmed present in the response:

(Figure 4.4.1 - Full curl -I output showing all security headers present)

Content-Security-Policy: default-src 'self';script-src 'self';style-src 'self' 'unsafe-inline';img-src 'self' data:;connect-src 'self';font-src 'self'; object-src 'none';frame-src 'none'
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
X-Permitted-Cross-Domain-Policies: none
X-Download-Options: noopen